AfyaTrust: Complete Project Flow

System guide · October 2026

This guide follows the current repository implementation from setup and account creation through records, consent, referrals, emergency access, audit, and deployment. It describes the proof-of-concept architecture in this repository; it is not a production deployment specification.

1. Project at a glance

AfyaTrust is a web application for managing patient records and access across health facilities. Its main parts are:

The frontend calls the Django API over HTTP. Django reads and writes PostgreSQL and, for chain-backed operations, calls the contract through Web3.py and a Sepolia JSON-RPC provider. The Angular app also includes optional Ethers.js wallet support, but the normal patient consent and API transaction path is backend-mediated.

```
Patient / doctor / hospital staff
              |
              v
      Angular web application
              | REST + JWT
              v
       Django REST API <------> PostgreSQL
              |
              | Web3.py + Sepolia RPC
              v
       AfyaTrust contract

Hardhat compiles, tests, deploys, and manages contract roles.
```

Hospital A and Hospital B are represented as facility identities in the same application and database. They are not separate installations in this PoC.

2. Data placement and identity

Stored in PostgreSQL

- Patient Health ID, name, phone, PIN hash, wallet address, and encrypted custodial private key.
- Doctor license number, approval status, facility, wallet address, and encrypted key.
- Hospital and staff profiles.
- Clinical record JSON, measurements, referral reasons, access request reasons, and detailed emergency reasons.
- Transaction references and local bookkeeping for access grants and referrals.

Stored or emitted on-chain

- A patient wallet and a stable keyed pseudonymous patient reference.
- Record hashes, facility identifiers, metadata pointers, and timestamps.
- Time-limited doctor grants, emergency access state, and role assignments.
- Events for grants, revocations, record views, referrals, and break-glass actions.

The API derives a reference as `pt_` plus HMAC-SHA-256 of a normalized Health ID using `CHAIN_ID_SECRET`. The Health ID is used to look up the patient in the application database; contract calls use the derived reference. This is pseudonymization, not anonymity: wallet addresses, facility identifiers, timestamps, and activity are public on a public chain. The old Sepolia deployment already contains Health IDs and names and cannot be scrubbed.

Clinical content stays in the database. The API computes a SHA-256 digest of a record payload and uses that digest to check integrity against the chain. A hash helps detect changes; it does not encrypt data or conceal predictable payloads by itself.

3. People, roles, and account lifecycle

Patient

1. The patient registers with Health ID, name, and PIN (or a facility submits the registration).
2. The backend generates a custodial Ethereum wallet. It encrypts the private key using AES-256-GCM with `MASTER_KEY` and stores the ciphertext and IV.
3. The backend derives the patient's pseudonymous reference and attempts `registerPatient` using the facility wallet.
4. The patient row is saved in PostgreSQL with a hash of the PIN. The API reports whether the chain transaction succeeded or is pending.
5. Patient login uses Health ID and PIN. Successful login returns access and refresh JWTs with the patient role.

Doctor

1. An administrator creates the doctor account, assigns a facility, sets the initial PIN, and approves the account.
2. The doctor has a custodial wallet. To use break-glass, the wallet must also have `CLINICIAN_ROLE` on the contract; the project includes a management command and Hardhat role scripts for this setup.
3. The doctor signs in through the shared login page with license number and PIN. Only approved doctors can sign in and use doctor workflows.

Hospital staff

1. An administrator creates the hospital and a staff account associated with that hospital.
2. Staff sign in through the same login endpoint using username and password.
3. The returned JWT includes the staff role and hospital identity. The API scopes referral and exchange operations to that hospital.

The UI has one login route. Angular routes then guard the patient, doctor, and hospital workspaces by role. These frontend guards improve navigation; Django JWT authentication and authorization remain the security boundary for API requests.

4. Shared request and transaction pattern

For a normal protected request:

1. Angular sends a request to the API, usually with `Authorization: Bearer <access-token>`.
2. Django validates the JWT, identifies the role, and loads the related patient, doctor, or staff profile.
3. The view validates the request and enforces role, ownership, facility, or consent rules.
4. Django reads/writes PostgreSQL, and calls the contract when the workflow requires chain state or an audit anchor.
5. The response returns JSON, status, and where available a transaction hash or a pending/error state.
6. Angular updates the appropriate workspace and refreshes affected data.

The backend blockchain adapter handles provider and contract setup, patient reference derivation, transaction signing, gas funding, role checks, and short-lived read caching. The facility wallet is used for facility/operator transactions. For custodial patient actions, the backend decrypts the patient key for the operation, while the contract verifies the supplied patient wallet and required role.

5. Main patient and clinical flows

5.1 Login and dashboard

The patient signs in with Health ID and PIN. The API returns a JWT. The Angular patient workspace loads the profile, dashboard summaries, records, measurements, requests, referrals, permissions, wallet activity, and activity history from patient endpoints. The API derives the patient identity from the JWT rather than trusting a Health ID supplied by the browser for patient-owned operations.

5.2 Add a clinical record

1. An authenticated facility user submits patient Health ID, record type, and clinical JSON to `POST /api/add-record/`.
2. The API resolves the patient and computes a SHA-256 hash over the canonical record payload.
3. The record payload and hash are stored in PostgreSQL.
4. The API calls `addRecord(patientRef, hash, facilityID, metadataURI)` on the contract using the facility wallet.
5. On success, the transaction hash is saved with the record. If the chain step fails, the API surfaces a pending/unanchored result; the stored row must not be presented as chain-verified.

The public exchange endpoint exposes verification metadata by record hash, not the clinical JSON.

5.3 Doctor requests access; patient decides

1. A doctor submits a patient's Health ID and a reason to `POST /api/doctor/request-access/`.
2. The API checks that the doctor exists and is approved, resolves the patient, and creates a pending `AccessRequest` in PostgreSQL.
3. The patient sees the request in the Requests workspace and can approve or reject it.
4. Approval submits `patientGrantAccess` through the facility operator, passing the patient's wallet and doctor's wallet. The contract checks the patient and clinician roles, then stores an expiry.
5. The API records a local `AccessGrant` for dashboard bookkeeping and returns the transaction hash. If the transaction is not confirmed, the API reports failure/pending; it must not imply that chain access was granted.
6. Rejection updates the request status without granting chain permission.

Patients can also grant access directly from the Permissions workspace, with a selected doctor and duration. Active access is determined by live contract state; local grant rows support lists and history when the provider is temporarily unavailable.

5.4 Doctor views a record

1. The doctor requests `GET /api/doctor/patient/<health_id>/`.
2. The API verifies the doctor JWT and approval, finds the patient, and checks `hasAccess` on-chain. Normal grants and an active emergency window can authorize access.
3. The API loads the patient's chain record anchors and matches them to database records by hash and facility.
4. It recomputes hashes from the clinical payload and releases only matching records. Missing or changed records are reported as integrity issues.
5. The API submits a best-effort asynchronous `recordView` audit transaction.
6. The doctor workspace displays verified data and any integrity or chain status.

If the chain cannot be checked, routine access does not release the medical history. The documented emergency behavior can return an explicit unverified-emergency state; that must not be confused with a normal verified read.

5.5 Measurements

1. A doctor submits a measurement to `POST /api/doctor/measurements/`.
2. The API requires current patient access, validates the numeric value, stores the reading and notes in PostgreSQL, and hashes the measurement payload.
3. The API attempts to anchor the hash and facility on-chain, then stores the transaction or pending status.
4. Patient and authorized doctor views can retrieve measurement history.
5. The patient integrity check can recompute measurement hashes and compare them with chain anchors.

Measurements are stored separately from `MedicalRecord`. The current inter-hospital referral verification endpoint does not include these measurement rows in its released referral packet.

5.6 Revoke normal access

The patient selects a doctor and calls `POST /api/patient/revoke-access/`. The API submits `patientRevokeAccess`, invalidates cached access state, and marks local grants inactive after confirmed revocation (or when chain state shows no live grant). If a live chain grant remains because the transaction failed, the API reports the failure and leaves the local grant active so the doctor is not falsely shown as revoked.

6. Referral and inter-hospital flow

Referral creation

- A doctor creates a referral for a patient and destination hospital, or a patient requests a referral from their workspace.
- The API stores the patient, source doctor/facility, destination, reason, status, and timestamps in PostgreSQL. Referral creation alone does not give destination doctors permission to read the patient's record.

Receiving hospital response

1. Hospital staff use incoming referral views scoped to their hospital.
2. Staff accept or decline using `POST /api/staff/referrals/<id>/respond/`.
3. The API updates the database status and responder information.
4. For acceptance, it attempts `logReferralAccepted` on-chain and records the transaction or pending chain error state.

Verify and receive records

1. Staff select an accepted referral and call `POST /api/staff/referrals/<id>/verify-records/`.
2. The API checks that the request belongs to the receiving hospital and that the referral is accepted.
3. It loads eligible source `MedicalRecord` rows and chain anchors for the patient reference.
4. It recomputes each payload hash and checks the pair of hash and facility against chain data.
5. Only matching records are included in the successful verification response. Mismatches or unavailable chain data are explicitly reported; unverified clinical content is withheld.
6. The receiving doctor's later access to the patient's full record still requires a patient grant or a valid break-glass window.

Referral acceptance is a transfer event, not a substitute for patient consent. Separate `Measurement` rows are not currently part of the referral verification response.

7. Emergency break-glass flow

1. An approved doctor submits patient Health ID and a documented emergency reason to `POST /api/doctor/break-glass/`.
2. The API verifies the doctor, patient, wallet role, and reason. It stores detailed reason information off-chain and hashes the reason.
3. The facility operator submits `breakGlass` with patient reference, doctor wallet, facility, and reason hash.
4. The contract grants a one-hour emergency access window and emits audit events. A local `AccessGrant` records the doctor, reason, expiry, and transaction status for the UI.
5. The doctor's record request follows the normal chain-access and integrity checks. The patient dashboard displays the active emergency access and countdown information.
6. The patient can end it early via `POST /api/patient/revoke-emergency/`; the operator submits `revokeEmergencyAccess` and the API updates local status after confirmation.
7. The emergency transaction remains part of immutable chain history after access expires or is revoked.

An emergency reason hash can prove a later-presented reason matches a prior commitment; it does not hide an incorrectly submitted reason. Do not submit the detailed reason itself on-chain.

8. Activity, integrity, and chain availability

The patient can review records, grants, revocations, referrals, views, and emergency events. When the provider is available, the API reads contract events and supplies transaction hashes for external verification. The activity-story endpoint can fall back to a database-derived narrative when chain history cannot be loaded. The UI identifies that source; a database story is not the same evidence as an on-chain event.

Chain availability affects operations differently:

The patient integrity endpoint re-hashes the current off-chain records and measurements and compares them with on-chain anchors. It provides a tamper check; it is not a backup or recovery mechanism.

9. Main API route groups

All routes below are under `/api/` unless noted. Protected endpoints require a JWT; the API remains responsible for role and object-level checks.

The route list in `backend/afyatrust/api/urls.py` is authoritative if this summary and the implementation ever differ.

10. Deployment and operations flow

Configure and start the backend

1. Create PostgreSQL database and credentials.
2. Copy `backend/.env.example` to `backend/.env`.
3. Set Django `SECRET_KEY`, DB settings, `MASTER_KEY`, and a random `CHAIN_ID_SECRET`; configure Sepolia RPC, contract address, and funded facility wallet for chain operations.
4. Install `backend/requirements.txt`, activate the Python environment, run `python manage.py migrate`, then start Django.
5. Create hospitals, doctors, and staff through the Django admin; approve doctors and ensure contract roles are correctly assigned.

Configure and start the frontend

1. Install dependencies in `frontend/` with npm.
2. Confirm the frontend environment API URL points to the backend and chain configuration points to the intended contract/network.
3. Run `npm start` for local development or `npm run build` for a production build.

Deploy a new contract

1. Copy `blockchain/.env.example` to `blockchain/.env`; set Sepolia RPC and an admin deployer key.
2. Compile, test, and deploy with Hardhat.
3. Configure backend contract address and grant the backend facility/operator roles. Grant clinician role only to verified doctors who need it.
4. Update the frontend's chain/address configuration and restart services.
5. Verify the chain ID and deployed address from both frontend and backend configurations before registering real test users.

A new contract starts empty. For replacement deployments, export and migrate state using the scripts documented in `blockchain/README.md`; preserve the same `CHAIN_ID_SECRET`. The state export intentionally omits raw patient IDs and names, and event history is not copied to a replacement contract.

11. Important configuration and operational secrets

Never commit `.env` files. Use synthetic data during development. Public-chain transactions and addresses are permanent and visible.

12. Current scope and known limits

- This is a proof of concept using a shared API/database and Sepolia testnet.
- It does not establish production identity proofing, clinical governance, facility isolation, disaster recovery, or regulatory compliance.
- Public-chain data is permanent. Pseudonymous references reduce direct disclosure but do not make activity anonymous.
- Referral verification covers `MedicalRecord` entries, not separate `Measurement` rows.
- The chain is the live authority for permissions and anchors. Local database status and fallback activity must be read with their source and verification status in mind.
- A hash confirms equality with an anchored digest; it does not guarantee the medical data was correct when first entered.

13. Source map

- Frontend routes: `frontend/src/app/app.routes.ts`
- Frontend API calls: `frontend/src/app/services/api.service.ts`
- API endpoints: `backend/afyatrust/api/urls.py`
- Workflow views: `backend/afyatrust/api/views.py`
- Data models: `backend/afyatrust/api/models.py`
- Blockchain adapter and pseudonym: `backend/afyatrust/api/blockchain.py`
- Wallet encryption: `backend/afyatrust/api/wallet_manager.py`
- Django settings: `backend/afyatrust/afyatrust/settings.py`
- Contract: `blockchain/contracts/AfyaTrust.sol`
- Deployment and migration details: `blockchain/README.md`
- Architecture diagrams: `docs/architecture.md`